```bash
# ---------- Declarations ---------- #
#!/bin/bash
read varname # Read input into varname
# variable declaration explicitly no space around =
varname="hello"
varname=1
varname=$(command)
varname=$((1+2))

# ------------ Arguments ------------ #
$0 # Command
$1 # First argument
$# # Number of arguments

# ---------- Conditionals ---------- #
if [ $n -# $n ]             # $n is eq/ne/lt/le/gt/ge than $n
if [ "$var" ]               # $var is not empty
if [ "$var" = "hello" ]     # $var is equal to "hello"
if [ "$var" != "hello" ]    # $var is not equal to "hello"
if [ "$var" \> "hello" ]    # $var is sorted after "hello"
if [ "$var" \< "hello" ]    # $var is sorted before "hello"
if [ -e dir ]               # dir exists
if [ -d dir ]               # dir is a directory
if [ -f file ]              # file is a regular file
if [ -r file ]              # file is readable
if [ -w file ]              # file is writable
if [ -x file ]              # file is executable
if [ -s file ]              # file is not empty

if [ condition1 ]
then
    #
elif [ condition2 ]
then
    #
else
    #
fi

# ------ Piping and redirection ------ #
command1 | command2     # Pipe output of command1 to input of command2
command > file          # Redirect output of command to file
command < file          # Redirect input of command from file
command >> file         # Append output of command to file
command 2> file         # Redirect error of command to file
command < file1 > file2 # Redirect input from file1 and output to file2

# -------- String operations -------- #
s="hello"
${#s}      # Length of string
${s:1:4}   # Substring: "ello"
${s//o/a}  # Replace all o with a
${s/o/a}   # Replace first o with a
# The replacements are case sensitive

# ---------- Basic commands ---------- #
g++ input.cpp -o output # Compile input.cpp to output

echo "$varname world"   # hello world
echo '$varname world'   # $varname world

pwd             # Print working directory
ls              # List files in directory
ls -l           # List files in directory with details
ls -a           # List all files in directory
ls -al          # List all files in directory with details
ls *.txt        # List all txt files
cd path         # Change directory to path
mv old new      # Rename old to new
cp old new      # Copy old to new
cp -r old new   # Copy old to new recursively
rm file         # Remove file
touch file      # Create file
cat file        # Print file content
man command     # Manual for command

# --- File reading and modification -- #
chmod [u/g/o][+/-/=][r/w/x] file    # Change file permissions
chmod 777 file                      # Give all permissions to file
# for each digit, it is made up of the sum of numbers 4, 2, 1
# for each number used in the sum, the represents permission r, w, x
# the digits represents u, g, o

cat file                # File content
wc                      # Word count
wc -l                   # Line count
wc -w                   # Word count
wc -m                   # Character count
cut -d' ' -f1           # Gets only first column
sort                    # Sort alphabetically
sort -nr                # Sort numerically reversed
sort -t',' -k2          # Sort by second column with delimiter ','
uniq                    # Remove adjacent duplicate lines
uniq -c                 # Prefix each output line w/ count of repeat
grep 'text'             # Print lines containing 'text'
grep -E 'regex'         # Print lines matching regex
grep -oE 'regex'        # Print only matching text on differnet lines
head -n 5               # Print first 5 lines
tr '[:upper:]' '[:lower:]' # Convert uppercase to lowercase
tr -s '[:space:]' '\n'  # Replace spaces with newlines
sed 's/old/new/g'       # Replace old with new
awk '{print $1}'        # Print first column separated by space
awk -F',' '{print $2}'  # Print second column separated by comma
awk '/super/ {print $1}' # Print first column of lines containing 'super'
awk -v max="$max_count" '$1 == max {print $2}'
# Print second column of lines with first column equal to max_count

while read -r line; do
    for word in $line; do
        echo "$word"
    done
done < "myfile.txt"

# ------------ Examples ------------ #
cat file | uniq -c | sort -nr | head -n 5
# Top 5 most frequent lines with count prefix
s = "hi HI hi hi1"
echo $s | tr '[:upper:]' '[:lower:]' # hi hi hi hi1
echo $s | grep -oE 'hi' # hi\n hi\n hi
echo $s | grep -E 'hi' # hi HI hi hi1 (matches line)
echo $s | grep -oE '\bhi\b' # hi\nhi

# Get each word in the file
words=$(cat myfile.txt | tr -s '[:space:]' '\n')
# Count the occurrence of each word
word_counts=$(echo "$words" | sort | uniq -c)
# Sort the word counts in descending order
sorted_counts=$(echo "$word_counts" | sort -nr)
# Find the maximum count
max_count=$(echo "$sorted_counts" | head -n 1 | awk '{print $1}')
# Filter the words with the maximum count
frequent=$(echo "$sorted_counts" | awk -v max="$max_count" '$1 == max {print $2}')
# Sort the most frequent words alphabetically
sorted_words=$(echo "$frequent" | sort)
# Print the most frequent words
echo "$sorted_words"
```

```cpp
/* ---------- Declarations ---------- */
int a, b, c;                    // Multiple declarations
int a[10];                      // Array of 10 ints (a[0] through a[9])
int a[]={0,1,2};                // Initialized array (or a[3]={0,1,2}; )
int a[2][3]={{1,2,3},{4,5,6}};  // Array of array of ints
char s[]="hello";               // String (6 elements including '\0')
int func(int x, bool y)         // func accepting int, bool returning int

/* --------- Variable scope --------- */
"Vars are only accessible at levels at or lower than where they are declared";
void f(int a) {
    a = a*a
    { int a = 4; cout << ++a << " ";}
    cout << a << " ";}
int main() {
    int a = 1;
    {
        cout << a << " "; // 1
        int a = 2;
        f(a); // 5 4
        cout << a << " "; // 2
    }
    f(a); // 5 1
    return 0;}

/* ---------- Control flow ---------- */
if (x < 0) {                    // If statement
    cout << "Negative" << endl;
} else if (x == 0) {            // Else if statement
    cout << "Zero" << endl;
} else {                        // Else statement
    cout << "Positive" << endl;
}
while (x < 10) {                // While loop
    cout << x << endl;
    x++;
} // Prints 0 to 9
a = [1, 2, 3, 4, 5];
for (int i = 0; i < 5; i++) {   // For loop (increment)
    cout << a[i] << endl;
} // Prints 1 to 5
for (int i = 4; i ≥ 0; i--) {  // For loop (decrement)
    cout << a[i] << endl;
} // Prints 5 to 1

/* ------------ Numbers ------------ */
b = i++; // b = i, i = i + 1
b = ++i; // i = i + 1, b = i
n = 1.5 / 2; // n = 0.75 (float division)
n = 2 / 1.5; // n = 1 (int division)
n = 2.9 % 1.5; // n = 1.4 (float remainder)
n = 2 % 1.5; // n = 0 (int remainder)

/* ---------- Characters ---------- */

char c = 'a';                   // Character
c = 97;                         // ASCII value of 'a'
c = c - 'a' + 'A';              // Convert to uppercase
c = c + 'a' - 'A';              // Convert to lowercase
c = '0' + 5;                    // Convert digit to char ('5')
isalpha(c), isdigit(c);          // Check if c is letter, digit
tolower(c), toupper(c);          // Convert to lowercase, uppercase

/* ------------ Libraries ---------- */
// # include <iostream>
cin >> x >> y;                  // Read words x and y (any type) from stdin
cout << "x=" << x << endl;      // Write line to stdout
// #include <iomanip>
cout << setprecision(2) << fixed << 3.14159; // Write 3.14 to stdout
cout << 12 << setw(6) << 34;    // Write "12    34" to stdout
cout << right << setw(6) << 34;  // Write "    34" to stdout
cout << left << setw(6) << 34;   // Write "34    " to stdout
cout << setfill('0') << setw(6) << 34; // Write "000034" to stdout
cout << scientific << 3.14159;  // Write 3.14159e+00 to stdout
cout << showpoint << 3.0;       // Write 3.0 to stdout
// #include <cstlib>
(rand () % N) + 1;              // Random number between 1 and N
rand() % (n - m + 1) + |m|;     // Random number between m and n
abs (-5);                       // Absolute value (5)
stoi("-123");                   // Convert string to int (-123)
stof("3.14");                   // Convert string to float (3.14)
// #include <algorithm>
sort(a, a+n);                   // Sort array a of size n
reverse(s.begin(), s.end());    // Reverse string s
replace(s.begin(), s.end(), 'a', 'b'); // Replace all 'a' with 'b' in s
// #include <string>
string s1, s2="hello";          // Create strings
s1.size(), s2.size();           // Number of characters: 0, 5
s1 += s2 + ' ' + "world";       // Concatenation
s = "hello" + " world";         // Error
s1 == "hello world";            // Comparison, also <, >, !=, etc.
s1[0];                          // 'h'
s1.substr(m, n);                // Substring s1[m,m+n]
s1.c_str();                     // Convert to const char*
s1.erase(m, n);                 // Erase s1[m,m+n]
s1.find(s2)                     // Check if s2 is in s1
s1.find(s2, m)                  // Check if s2 is in s1 starting at m
s1.rfind(s2)                    // Check if s2 is in s1 from the end
getline(cin, s);                // Read line ending in '\n'
s1.insert(m, s2, m2, n2)        // Insert s2[m2,m2+n2] into s1[m]
to_string(123);                 // Convert int to string ("123")

/* ------- String comparisons ------- */
"String comparisons are carried out character by character, by ANSCII val";
string msg1 = "Apple", msg2 = "apple";
string msg3 = "apples", msg4 = "orange";
bool c1 = msg1 == msg2;         // false
bool c2 = msg3 != msg4;         // true
bool c3 = msg1 < msg2;          // true (A < a)
bool c4 = msg2 < msg3;          // true (0 < s)
bool c5 = msg4 > msg3;          // true (o > a)

/* ---------- Lib: Streams ---------- */
// #include <fstream>
ifstream fin("input.txt");          // Declare fin
ofstream fout("output.txt");        // Delcare fout
fin.open("input.txt");              // Open file for reading
fout.open("output.txt");            // Open / create file for writing
fout.open("output.txt", ios::app);  // Open file for appending

while (fin >> x) {                  // Read words from file
    cout << x << endl;
} // Prints words from file

while (getline(fin, line)) {        // Read lines from file
    cout << line << endl;
} // Prints lines from file
fout << "Hello, world!" << endl;    // Write to file
fin.close(); fout.close();          // Close file

// sstream
string str = "123 456 789";
istringstream iss(str);     // Read from string
while (iss >> x) {          // Read words from string
    cout << x << endl;
} // Prints 123\n456\n789

/* ----- Lib: Vector, List, Maps ---- */
// #include <vector> or <list>
list<int> l;                    // List of ints
vector<int> v;                  // Vector of ints
v.size();                       // Number of elements
v[0];                           // First element
v.begin(); v.end();             // Iterator to first, past last element
v.push_back(5);                 // Add element to end
v.pop_back();                   // Remove last element
v.pop_front();                  // Remove first element
v.erase(v.begin() + 1);         // Remove second element
v.erase(v.begin(), v.begin() + 2); // Remove first two elements
v.clear();                      // Remove all elements
sort(v.begin(), v.end());       // Sort elements

// #include <map> (Maps are not ordered by insertion order)
map<string,int> m;              // Map from string to int
m = {{"a",1},{"b",2}};          // Initialize map
m["a"];                         // Access value
m["c"] = 3;                     // Insert value
m.count("a");                   // Number of elements with key "a"
m.erase("a");                   // Remove element with key "a"

vector<int>::iterator it;       // Iterator for vector of ints
for (itr = v.begin(); itr != v.end(); itr++) {
    cout << *itr << endl;
} // Iterate through vector

map<string,int>::iterator it;   // Iterator for map of string to int
for (it = m.begin(); it != m.end(); it++) {
    cout << it->first << " " << it->second << endl;
} // Iterate through map

/* ------------ Structs ------------ */
struct Point { int x, y; };     // Define struct
Point p; p.x = 1; p.y = 2;       // Create instance with dot operator
Point p = {1, 2};                // Create instance

bool operator <(const Point& a, const Point& b) {
    return a.x < b.x || (a.x == b.x && a.y < b.y);
} // Define comparison operator

/* ----- Pointers and References ---- */
int v = 5;
int* p;             // declares a pointer to int
p = &v;             // stores address of v into p (p is a pointer to int)
int& r = v;         // r is an alias of v
cout << v << ", " << r << endl;     // prints 5, 5
cout << &v << &r << endl;           // prints address of v twice
cout << *p << endl;                 // dereferences p to get value of v
int* p; p = new int; *p = 5;        // allocates memory for int, stores 5

int* p;
delete p; // Error: p is uninitialized
cout << *p; // Error: p is a dangling pointer
p = '5'; // Error: p is a pointer to int
p = 5; // Error: p is a int pointer type not int type

char x[10];
char* p = x;            // p points to the first element of x
p++;                    // p points to the next element of x
cout << p[-1] << endl;  // prints the first element of x
cout << *p << endl;     // prints the second element of x
cout << *(p+1) << endl; // prints the third element of x

void swap(int& a, int& b) { // Pass by reference
    int temp = a; a = b; b = temp;
} // Swaps a and b

void swap(int* a, int* b) { // Pass by pointer
    int temp = *a; *a = *b; *b = temp;
} // Swaps a and b

void func(int p[])      // if p is modified, the original is affected

/* -------- Memory allocation ------- */
```

```cpp
int* p = new int (5);   // allocate memory for an int and initialize to 5
delete p;               // deallocate memory
int* dynamicArray = new int[size];
delete [] dynamicArray;

list<int>* func() {
    int* p = new list<int>;   // allocate memory for an int
    list->push_back(5);       // access with arrow operator
    return p;
}


/* ---------- Linked lists ---------- */
struct Node { int data; Node* next; };  // Define struct
Node* head = nullptr;                   // Empty linked list (no nodes)
Node* n = new Node;                     // Create empty note
n->data = 5; (*n).data = 5;             // Arrow vs dot (equiv)
n.data = 5;                             // Error: n is a pointer
head = n;                               // Insert at front

Node* current = head;                   // Traverse linked list
while (current != nullptr) {            // Loop until end of list
    cout << current->data << endl;
    current = current->next;
} // Prints all elements

void insertAtFront(Node*& head, int data) { // Insert at front
    Node* n = new Node;             // Initialize new node
    n->data = data; n->next = head; // Initialize new node
    head = n;                       // Insert at front
} // Node*&: *  pass by refernece,  & allows changing the head

void insertAtFrnt(Node** head, int data) {
    Node* n = new Node;
    n->data = data; n->next = *head;
    *head = n;
} // Same but using **

void insertAtBack(Node*& head, int data) { // Insert at back
    Node* n = new Node;             // Initialize new node
    n->data = data; n->next = nullptr; // Initialize new node
    if (head == nullptr) { head = n; return; } // Insert at front
    Node* current = head;           // Traverse list
    while (current->next != nullptr) { current = current->next; } // Find end
    current->next = n;              // Insert at back
}

void doSomethingAfterFinding(Node*& head, int data) { // Find node with data
    Node* current = head;                             // Initialize pointer
    while (current != nullptr) {                      // Traverse list
        if (current->data == data) {
            // do something
            return;
        }
        current = current->next;}}

void remove(Node*& head, int data) { // Remove first node with data
    Node* current = head, *prev = nullptr;
    while (current != nullptr) {
        if (current->data == data) {
            if (prev == nullptr) head = current->next;
            else prev->next = current->next;
            delete current;
            return;
        }
        prev = current;
        current = current->next;}}

void reverse(Node*& head) { // Reverse linked list
    Node* prev = nullptr, *current = head, *next; // Initialize pointers
    while (current != nullptr) { // Traverse list
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    head = prev;}

void deleteList(Node*& head) { // Delete linked list
    Node* current = head, *next; // Initialize pointers
    while (current != nullptr) { // Traverse list
        next = current->next;
        delete current;
        current = next;
    }
    head = nullptr;}

void removeDups(Node*& head) { // Remove duplicates
    Node* current = head, *prev = nullptr; // Initialize pointers
    set<int> s; // Initialize set
    while (current != nullptr) { // Traverse list
        if (s.count(current->data)) { // Check if duplicate
            prev->next = current->next; // Remove duplicate
            delete current;
        } else {
            s.insert(current->data); // Add to set
            prev = current;
        }
        current = prev->next;}}

/* -------------- Misc -------------- */
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);}}}}

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        swap(arr[i], arr[minIndex]);}}

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j ≥ 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;}}

void reverseArray(int arr[], int n) {
    for (int i = 0; i < n / 2; i++) {
        swap(arr[i], arr[n - i - 1]);
    }}

int divisors(int n) {
    int count = 0;
    for (int i = 1; i ≤ n; i++) {
        if (n % i == 0) {
            count++;
        }
    }
    return count;}

/* ------- Doubly Linked List ------- */
struct Node {int data;Node* next;Node* prev;};

void insertAtFront(Node*& head, int data) {
    Node* n = new Node;
    n->data = data;
    n->next = head;
    n->prev = nullptr;
    if (head != nullptr) {
        head->prev = n;
    }
    head = n;}

void insertAtBack(Node*& head, int data) {
    Node* n = new Node;
    n->data = data;
    n->next = nullptr;
    if (head == nullptr) {
        n->prev = nullptr;
        head = n;
        return;
    }
    Node* current = head;
    while (current->next != nullptr) {
        current = current->next;
    }
    current->next = n;
    n->prev = current;}

void remove(Node*& head, int data) {
    Node* current = head, *prev = nullptr;
    while (current != nullptr) {
        if (current->data == data) {
            if (prev == nullptr) {
                head = current->next;
            } else {
                prev->next = current->next;
            }
            if (current->next != nullptr) {
                current->next->prev = prev;
            }
            delete current;
            return;
        }
        prev = current;
        current = current->next;}}

void reverse(Node*& head) {
    Node* current = head, *temp = nullptr;
    while (current != nullptr) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }
    if (temp != nullptr) {
        head = temp->prev;}}

void removeDups(Node*& head) {
    Node* current = head, *prev = nullptr;
    set<int> s;
    while (current != nullptr) {
        if (s.count(current->data)) {
            prev->next = current->next;
            if (current->next != nullptr) {
                current->next->prev = prev;
            }
            delete current;
        } else {
            s.insert(current->data);
            prev = current;
        }
        current = prev->next;}}

/* ---------- Recursion ---------- */
"Recursion is a function that calls itself";
"Recursion must have a base case to stop the recursion";

int binarySearch(int arr[], int l, int r, int x) {
    if (r ≥ l) { // Base case
        int mid = l + (r - l) / 2; // Calculate mid
        if (arr[mid] == x) return mid; // Found x
        if (arr[mid] > x) return binarySearch(arr, l, mid - 1, x); // Search left
        return binarySearch(arr, mid + 1, r, x); // Search right
    }
    return -1; // Not found
}

int bubbleSort(int arr[], int n) {
    if (n == 1) return; // Base case
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) swap(arr[i], arr[i + 1]);
    }
    bubbleSort(arr, n - 1); // Recurse
}

int selectionSort(int arr[], int n) {
    if (n == 1) return; // Base case
    int minIndex = 0;
    for (int i = 1; i < n; i++) {
        if (arr[i] < arr[minIndex]) minIndex = i;
    }
    swap(arr[0], arr[minIndex]);
    selectionSort(arr + 1, n - 1); // Recurse
}

int insertionSort(int arr[], int n) {
    if (n ≤ 1) return; // Base case
    insertionSort(arr, n - 1); // Recurse
    int key = arr[n - 1], j = n - 2;
    while (j ≥ 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}

int palindrome(string s, int l, int r) {
    if (l ≥ r) return 1; // Base case
    if (s[l] != s[r]) return 0; // Not a palindrome
    return palindrome(s, l + 1, r - 1); // Recurse
}


"Compared to C++, C does not have:";
// Classes and objects
// New/delete operators (use malloc/free instead)
// Namespace
// String class
// Pass by reference (use pointers instead)

/* ------------ I/O in C ------------ */
#include <stdio.h>                          // Include standard I/O library
int x, y;                                   // Declare variables
gets(s);                                    // Read line from stdin
scanf("%d %d", &x, &y);                     // Read integers x and y from stdin
printf("x=%d\n", x);                        // Write line to stdout
"%d"    // Integer
".3%f"    // Float with 3 decimal places
"%g"    // No trailing 0 (scientific notation)
"%lf"   // Double
"%c"    // Character
"%s"    // String

/* ---------- Strings in C ---------- */
#include <string.h>                         // Include string library
char s[100];                                // Allocate memory for a string
char s[] = "hello";                         // Declare and initialize a string
char a[] = ['h', 'e', 'l', 'l', 'o', '\0']; // Same as above
strlen(s);                                  // Number of characters: 5
strcpy(s, "world");                         // Copy "world" to s
strcmp(s, "world");                         // <: -1, =: 0, >: 1
sprintf(s, "%d", 123);                      // Convert int to string
sprintf(s, "%s%d", "hello", 123);           // Concatenate string and int

/* - Dynamic memory allocation in C - */
#include <stdlib.h>                         // Include standard library
int *a = (int*) malloc(10 * sizeof(int));   // Allocate memory for 10 integers
// A is an array of 10 integers
free(a);                                    // Free memory

int main(int argc, char *argv[]) {}
// argc number of arguments (including program name)
// argv array of arguments


# ------------- Makefile ------------- #
gcd.o: gcd.cpp gcd.h
# target: dependencies
    g++ $(FLAGS) -c gcd.cpp # action
# Compiles gcd.cpp into gcd.o without linking
gcd_main.o: gcd_main.cpp gcd.h
    g++ $(FLAGS) -c $< # $< first dependency
gcd: gcd.o gcd_main.o # Links the object files
    g++ $(FLAGS) $^ -o $@
    # $^: all dependencies (gc.o gcd_main.o)
    # $@: target (gcd)
clean: # a shorthand function to clean up
    rm -f gcd gcd.o gcd_main.o gcd.tgz
.PHONY: clean tar # Declare shorthands
```

Symbol	Description	Symbol	Description
^	Start of line +	?	0 or 1 +
\A	Start of string +	{3}	Exactly 3 +
$	End of line +	{3,}	3 or more +
\Z	End of string +	{3,5}	3, 4 or 5 +
\b	Word boundary +	\	Escape Character +
\B	Not word boundary +	\n	New line +
\<	Start of word	\r	Carriage return +
\>	End of word	\t	Tab +
\s	White space	.	Any character except new line (\n) +
\S	Not white space	(a|b)	a or b +
\d	Digit	[abc]	Range (a or b or c) +
\D	Not digit	[^abc]	Not a or b or c +
\w	Word	[0-7]	Digit between 0 and 7 +
\W	Not word	[a-q]	Letter between a and q +
*	0 or more +	[A-Q]	Upper case letter + between A and Q +
+	1 or more +		